192525074_Shaik Ayesha Tabassum_CO1_AT2

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO1: *Analyse DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships.* (BL1, BL2)

Activity Tool : Concept Mapping (Medium)

Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Construct a concept map linking the following terms: DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship.	BL2 – Understand
2	Create a concept map for the three-level ANSI/SPARC architecture showing data independence at each level.	BL2 – Understand
3	Map the relationship between file-based systems and database systems, highlighting the problems solved by DBMS.	BL2 – Understand
4	Design a concept map for ER diagram components: entity, weak entity, attribute types, relationship types, and participation constraints.	BL2 – Understand
5	Construct a concept map comparing Hierarchical, Network, and Relational data models with their key properties.	BL2 – Understand
6	Develop a concept map for the EER model showing how it extends the basic ER model with generalization, specialization, and aggregation.	BL2 – Understand

7	Create a visual concept map for the DBMS software architecture, illustrating the flow from user request to data retrieval.	BL2 – Understand
8	Map the data models discussed in CO1 to real-world applications and explain the suitability of each model.	BL2 – Understand
9	Construct a concept map for schemas and instances, showing how they relate to physical and logical data independence.	BL2 – Understand
10	Design a concept map for the roles and responsibilities in a DBMS environment: DBA, End User, Application Developer.	BL1 – Remember

---

Code of Conduct:

I Shaik Ayesha Tabassum(192525074) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

S.Ayesha Tabassum

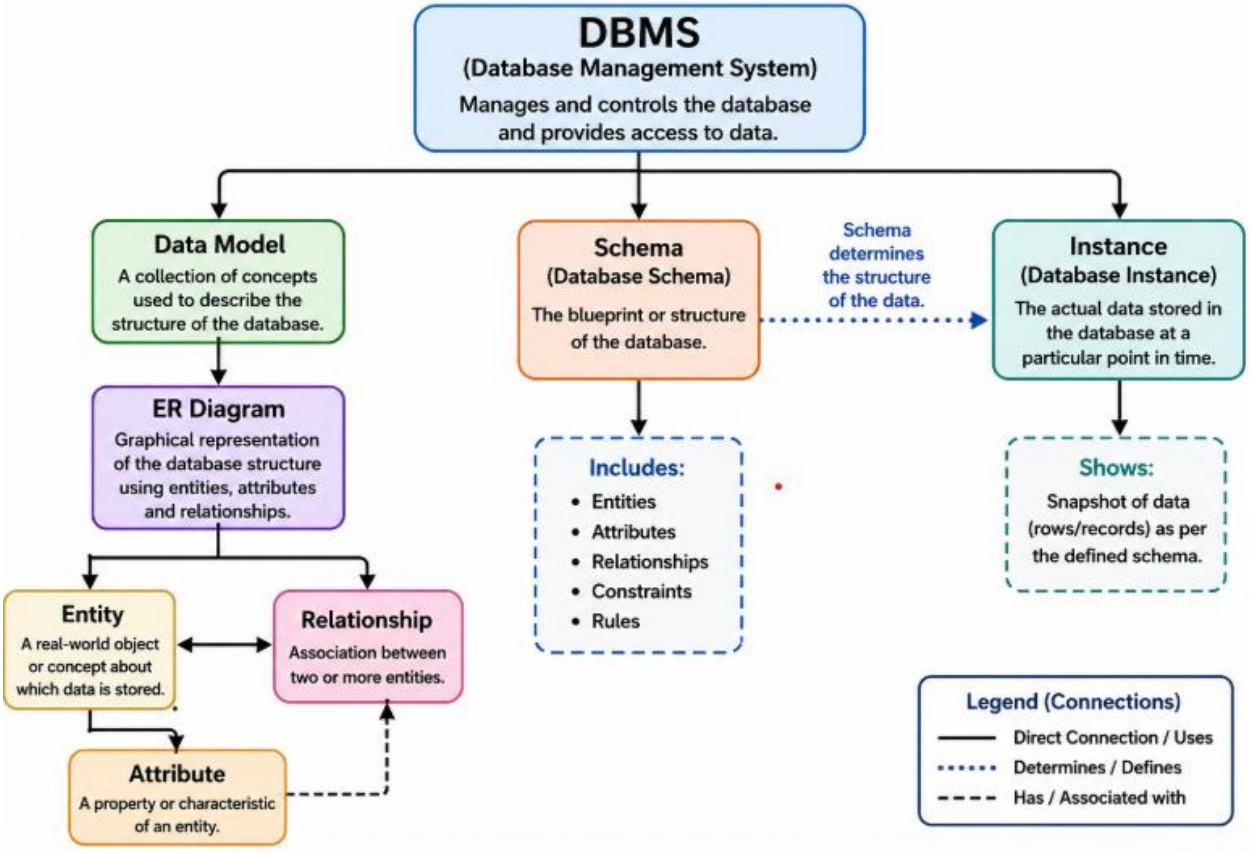
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure

Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

1. Construct a concept map linking the following terms: DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship.



DBMS (Database Management System)

A **Database Management System (DBMS)** is software that stores, manages, and retrieves data efficiently. It provides security, reduces data redundancy, and allows multiple users to access the database simultaneously.

Data Model

A **Data Model** defines how data is organized, stored, and related within a database. It provides the structure for designing databases.

Schema

A **Schema** is the logical blueprint of a database. It defines the tables, fields, relationships, and constraints, and usually remains unchanged unless the database structure is modified.

Instance

An **Instance** is the actual data stored in the database at a particular point in time. Unlike the schema, it changes whenever data is inserted, updated, or deleted.

ER Diagram (Entity-Relationship Diagram)

An **ER Diagram** is a graphical representation of a database. It illustrates entities, attributes, and relationships, helping designers visualize the database structure before implementation.

Entity

An **Entity** is any real-world object, person, place, or thing about which data is stored.
Examples: Student, Employee, Product, Customer.

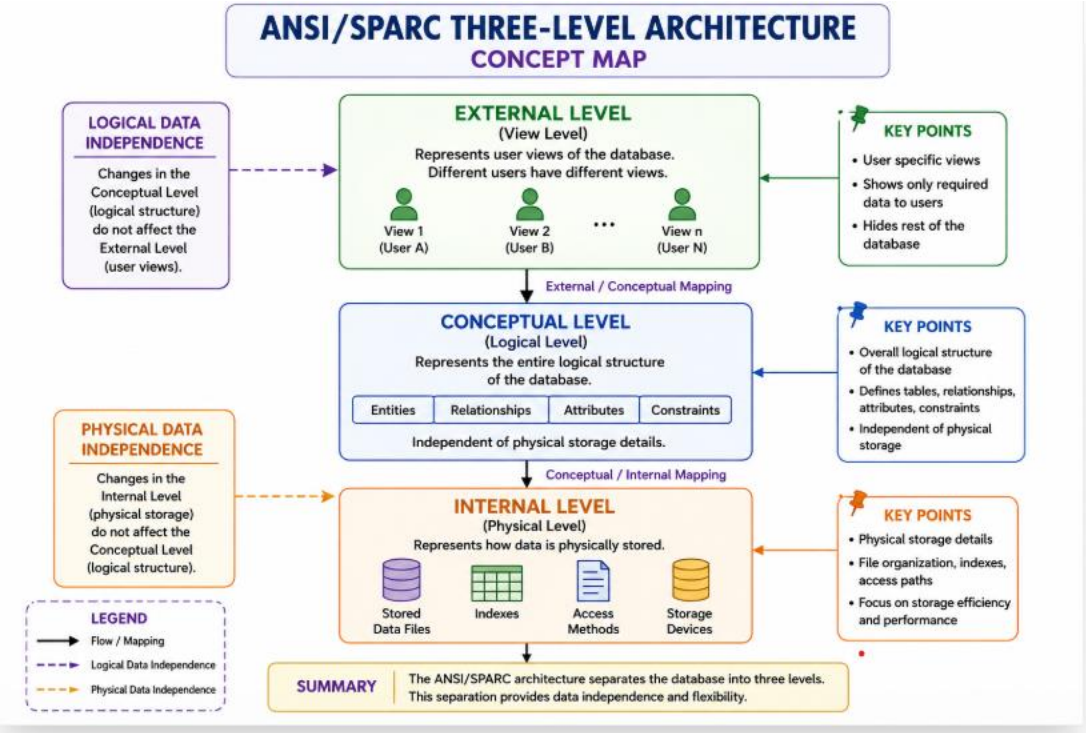
Attribute

An **Attribute** is a property or characteristic of an entity. It provides detailed information about that entity.
Example: For a Student entity, attributes can be Student_ID, Name, Department, and Age.

Relationship

A **Relationship** describes how two or more entities are connected. It shows the association between entities in a database.

2. Create a concept map for the three-level ANSI/SPARC architecture showing data independence at each level.



ANSI/SPARC Architecture

The ANSI/SPARC architecture is a **three-level database architecture** that separates user applications from physical database storage. It improves flexibility, security, and data independence.

1. External Level (View Level)

- Represents the user view of the database.
- Different users can access different views.
- Hides unnecessary data from users.

2. Conceptual Level (Logical Level)

- Represents the complete logical structure of the database.
- Defines tables, relationships, constraints, and data types.
- Independent of physical storage.

3. Internal Level (Physical Level)

- Represents how data is physically stored.
- Includes files, indexes, storage devices, and access methods.
- Focuses on storage efficiency and performance.

Data Independence

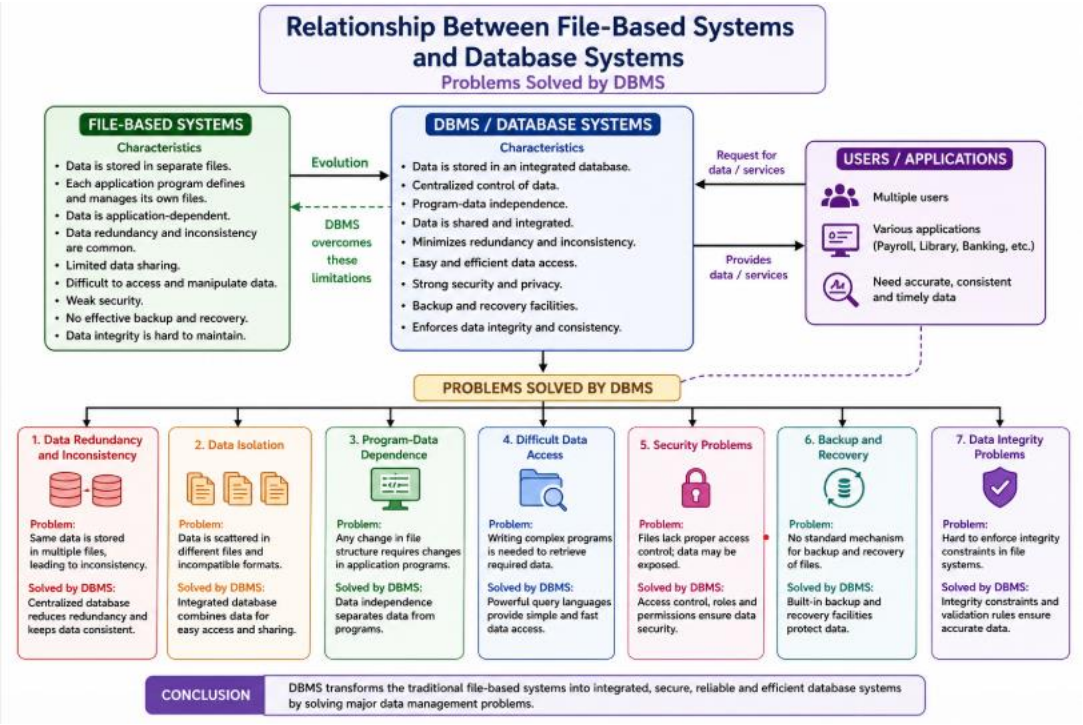
Logical Data Independence

- Ability to change the **Conceptual Level** without affecting the **External Level**.
- Example: Adding a new column to a table without changing user views.

Physical Data Independence

- Ability to change the **Internal Level** without affecting the **Conceptual Level**.
- Example: Changing file organization or adding indexes without changing the database schema.

3. Map the relationship between file-based systems and database systems, highlighting the problems solved by DBMS.



Relationship Between File-Based Systems and Database Systems (Short)

- **File-Based System:** Stores data in separate files, leading to redundancy, inconsistency, and poor security.
- **DBMS:** Stores data in a centralized database, making data management efficient and secure.

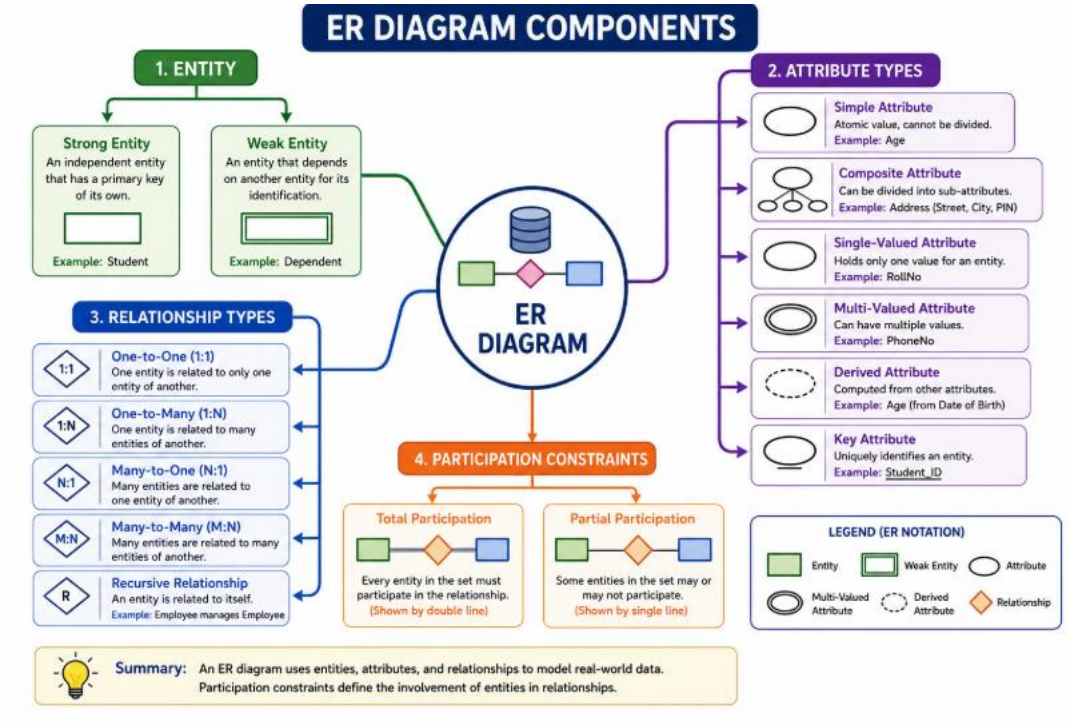
Problems Solved by DBMS

- Reduces **data redundancy**.
- Eliminates **data inconsistency**.
- Improves **data sharing**.
- Provides **better security**.
- Supports **backup and recovery**.
- Maintains **data integrity**.
- Enables **easy and fast data access**.

Conclusion

A **DBMS** solves the major limitations of file-based systems by providing centralized, secure, reliable, and efficient data management.

4. Design a concept map for ER diagram components: entity, weak entity, attribute types, relationship types, and participation constraints.



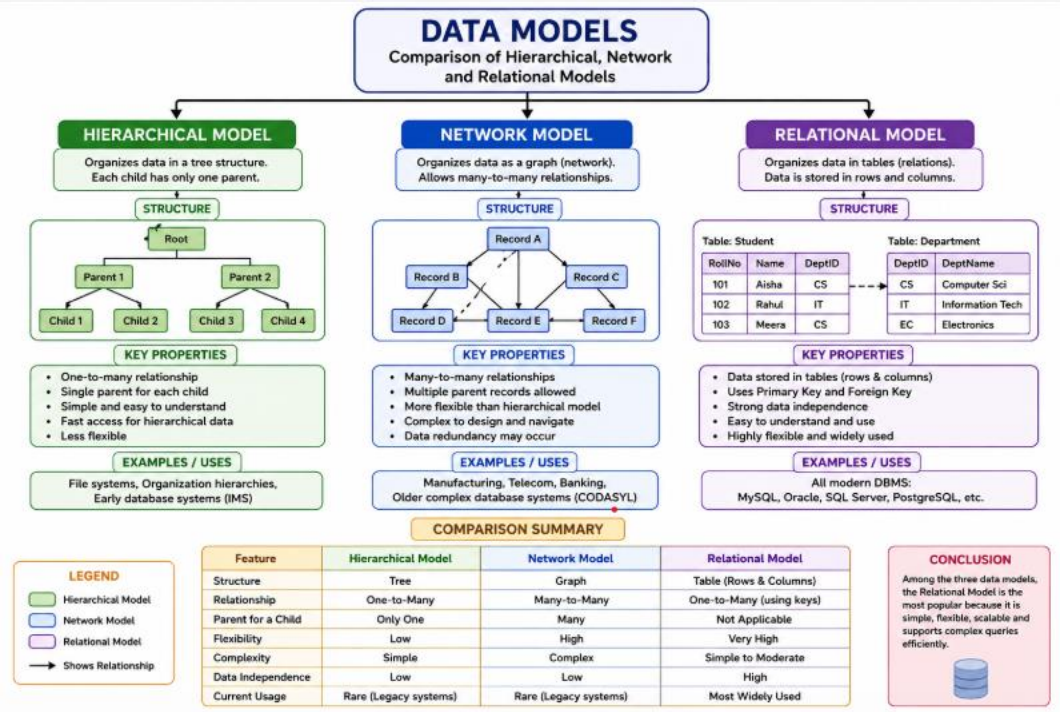
An **Entity-Relationship (ER) Diagram** is a graphical representation of a database that shows entities, attributes, relationships, and constraints. It helps in designing an efficient database structure.

- **Entity:** A real-world object or person about which data is stored (e.g., Student, Employee).
- **Weak Entity:** An entity that cannot exist without a related strong entity and depends on it for identification.
- **Attribute Types:** Describe the properties of an entity, such as **Simple, Composite, Single-Valued, Multi-Valued, Derived,** and **Key Attributes**.
- **Relationship Types:** Define how entities are connected, including **One-to-One (1:1), One-to-Many (1:N), Many-to-One (N:1), Many-to-Many (M:N),** and **Recursive Relationships**.
- **Participation Constraints:** Specify whether an entity's participation in a relationship is **Total** (mandatory) or **Partial** (optional).

Conclusion

ER diagram components help represent real-world data clearly and accurately, making database design simple, organized, and efficient.

5. Construct a concept map comparing Hierarchical, Network, and Relational data models with their key properties.



A **Data Model** defines how data is organized, stored, and related in a database. The three main data models are **Hierarchical, Network, and Relational**.

- **Hierarchical Model:** Organizes data in a **tree structure** where each child has only one parent. It is simple and suitable for one-to-many relationships.
- **Network Model:** Organizes data in a **graph structure** where a child can have multiple parents. It supports many-to-many relationships and provides greater flexibility.
- **Relational Model:** Organizes data in **tables (rows and columns)**. Tables are connected using **primary keys** and **foreign keys**, making it the most widely used and efficient data model.

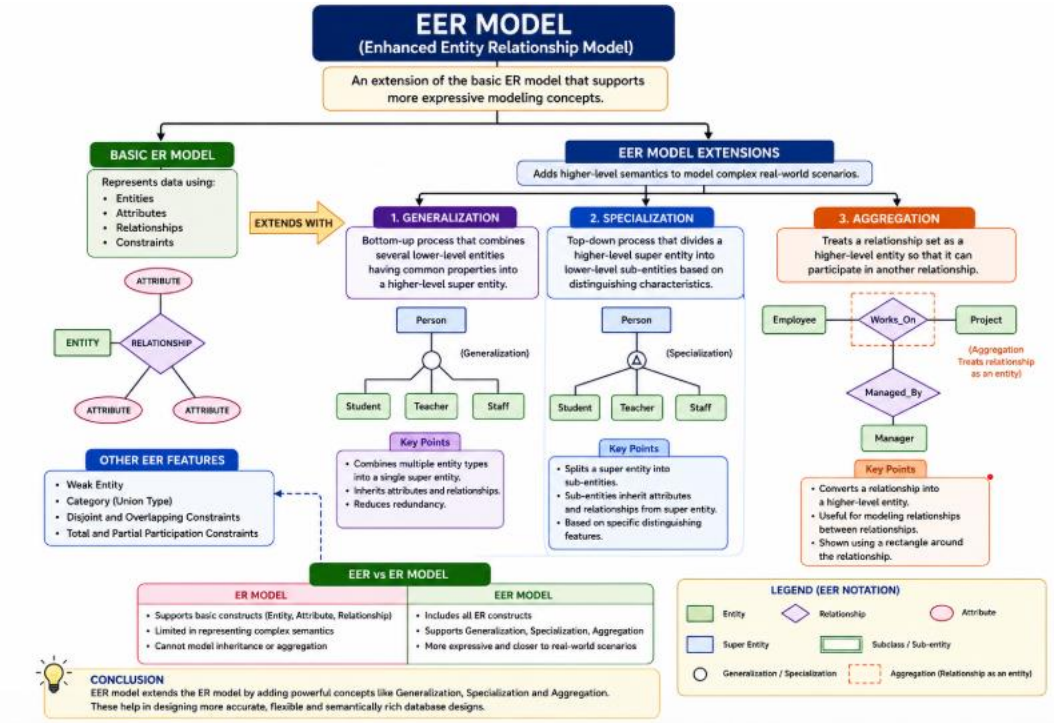
Key Properties

- **Hierarchical:** Tree structure, one-to-many relationship, simple but less flexible.
- **Network:** Graph structure, many-to-many relationship, flexible but complex.
- **Relational:** Table structure, key-based relationships, easy to use and widely adopted.

Conclusion

Among the three, the **Relational Model** is the most popular because it provides efficient data storage, easy querying, high flexibility, and better data management.

6. Develop a concept map for the EER model showing how it extends the basic ER model with generalization, specialization, and aggregation.



The **Enhanced Entity-Relationship (EER) Model** is an extension of the basic ER model. It provides advanced features to represent complex real-world data more accurately.

- **Generalization:** Combines similar lower-level entities into a common higher-level **superclass**.
- **Specialization:** Divides a higher-level entity into smaller **subclasses** based on specific characteristics.
- **Aggregation:** Treats a relationship as a higher-level entity so it can participate in another relationship.

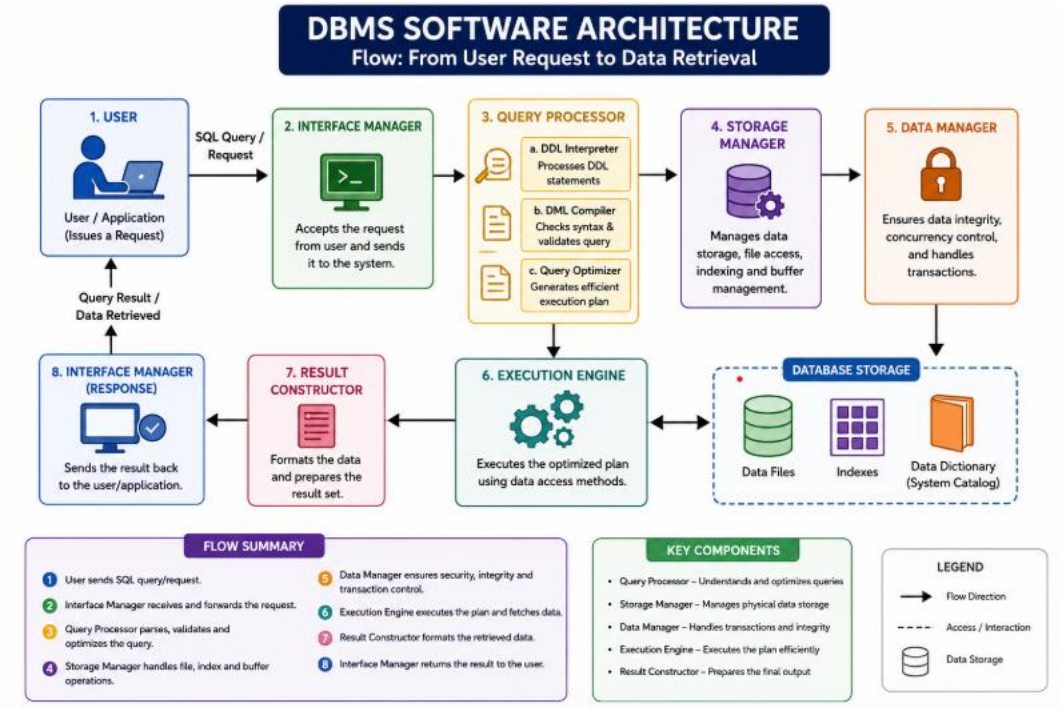
Key Points

- Extends the basic **ER Model**.
- Supports **inheritance** between entities.
- Represents complex relationships more effectively.
- Improves database design by reducing redundancy and increasing flexibility.

Conclusion

The **EER Model** enhances the ER model with **generalization, specialization, and aggregation**, making database design more powerful, flexible, and suitable for real-world applications.

7. Create a visual concept map for the DBMS software architecture, illustrating the flow from user request to data retrieval.



DBMS Software Architecture – Short Matter

The **DBMS Software Architecture** shows how a user's request is processed and converted into the required data. It ensures efficient, secure, and reliable data retrieval.

- **User Request:** The user submits a query or request through an application.
- **Query Processor:** Analyzes, validates, and optimizes the query for execution.
- **Storage Manager:** Manages data files, indexes, and physical storage.
- **Database:** Stores the actual data and system catalog.
- **Execution Engine:** Retrieves the required data from the database.
- **Result:** The processed data is returned to the user.

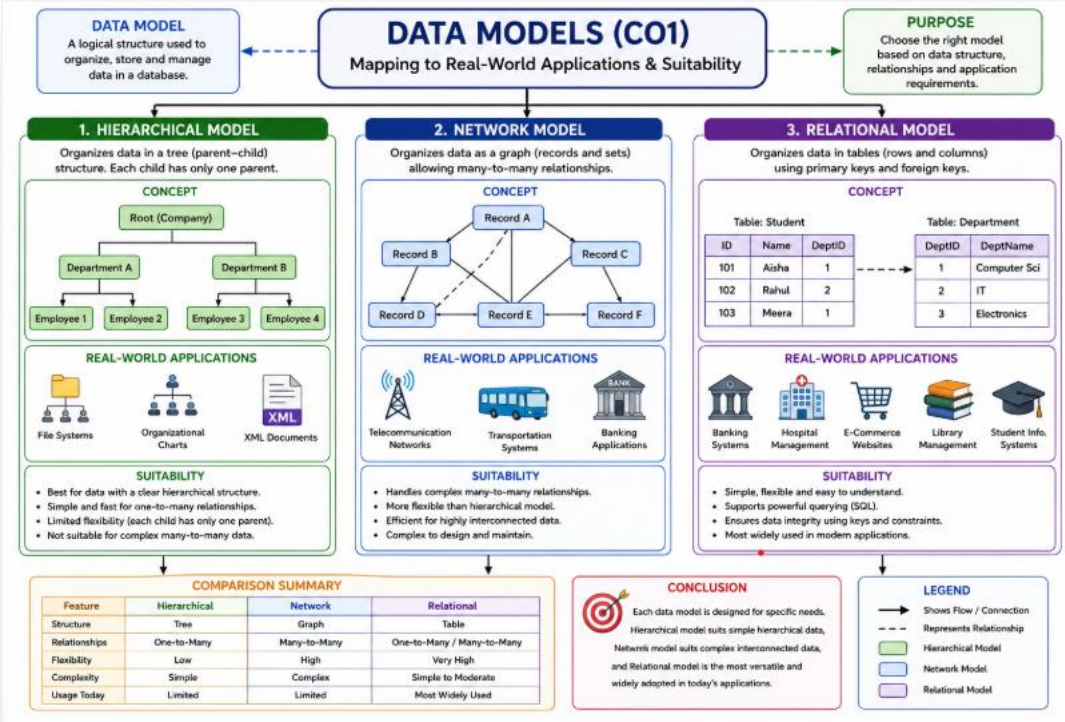
Flow

User Request → Query Processor → Storage Manager → Database → Data Retrieval → Result to User

Conclusion

The DBMS software architecture efficiently processes user requests, retrieves accurate data, and ensures security, integrity, and fast database operations.

8. Map the data models discussed in CO1 to real-world applications and explain the suitability of each model.



Data Models and Their Real-World Applications

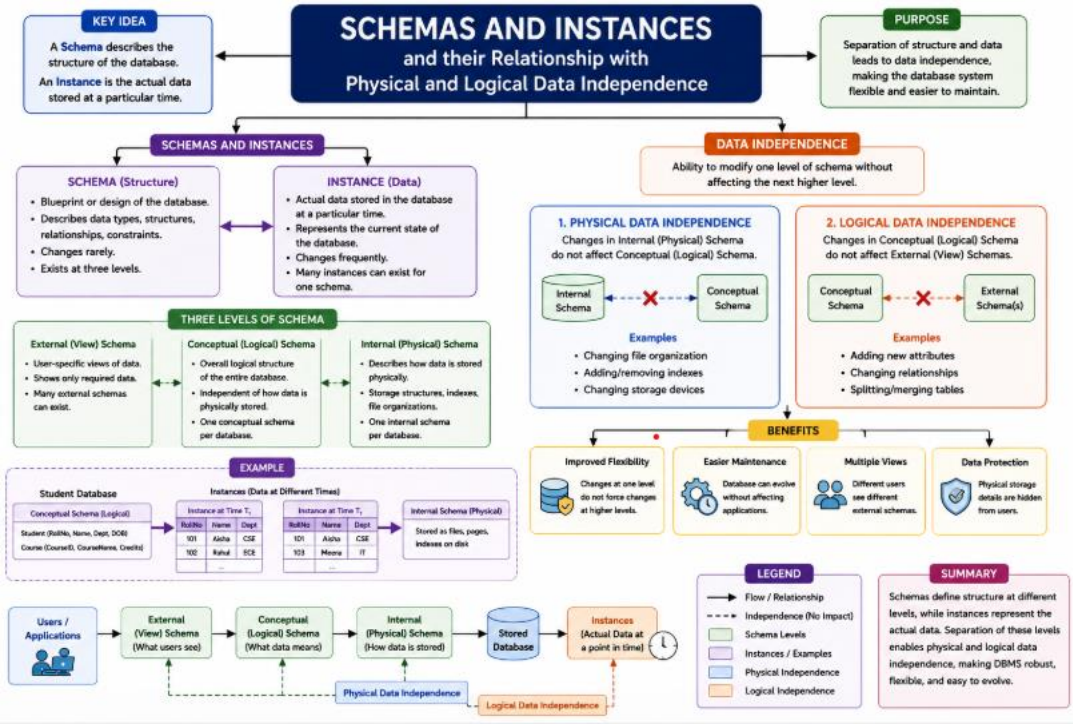
Data models define how data is organized and managed in a database. Different models are suitable for different types of applications.

- **Hierarchical Model:** Organizes data in a tree structure with a parent–child relationship. It is suitable for **file systems, organizational charts, and XML documents** where data follows a hierarchy.
- **Network Model:** Organizes data as a graph, allowing many-to-many relationships. It is suitable for **telecommunication networks, transportation systems, and banking applications**, where records have multiple connections.
- **Relational Model:** Organizes data in tables using rows, columns, and keys. It is suitable for **banking systems, hospital management, e-commerce, library management, and student information systems** because it provides flexibility, easy querying, and efficient data management.

Conclusion

Each data model is designed for specific applications. The **Hierarchical Model** is best for hierarchical data, the **Network Model** is suitable for complex interconnected data, and the **Relational Model** is the most widely used due to its simplicity, flexibility, and powerful data management capabilities.

9. Construct a concept map for schemas and instances, showing how they relate to physical and logical data independence.



Schemas and Instances with Data Independence

A **Schema** is the overall design or blueprint of a database. It defines the structure of the database, including tables, attributes, relationships, and constraints. Since the schema describes the organization of data, it changes very rarely.

An **Instance** is the actual data stored in the database at a specific point in time. It represents the current state of the database and changes whenever records are inserted, updated, or deleted.

The concept of **Data Independence** allows changes to one level of the database without affecting other levels.

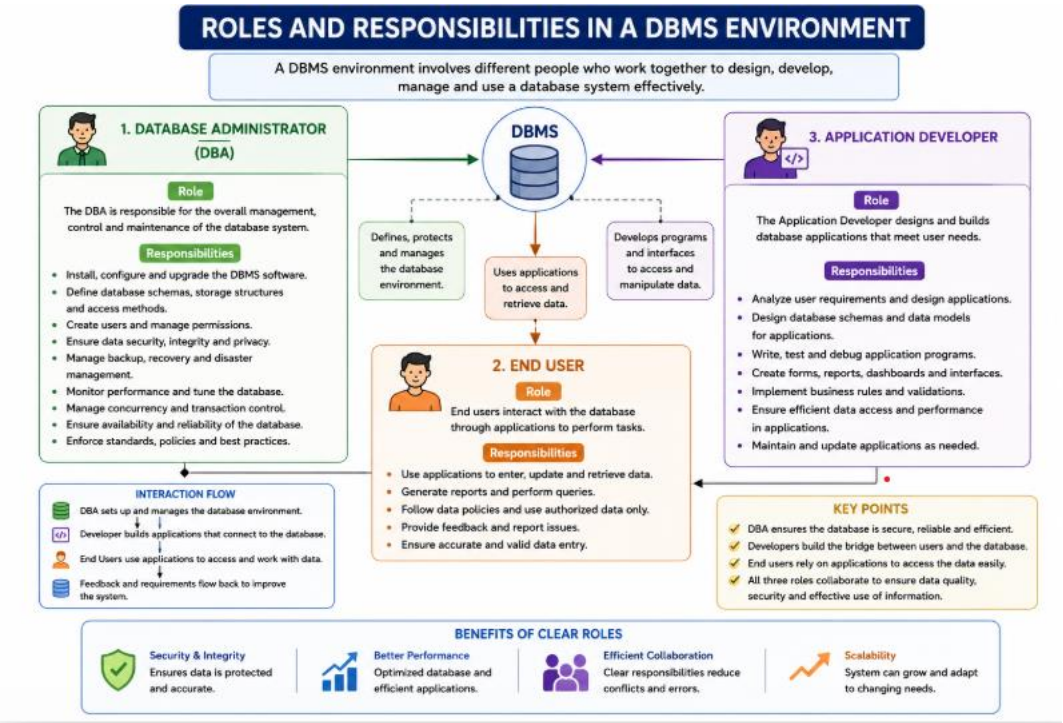
- **Physical Data Independence** is the ability to modify the physical storage of data, such as changing file organization or indexes, without affecting the logical schema or application programs.
- **Logical Data Independence** is the ability to modify the logical structure of the database, such as adding new attributes or tables, without affecting user views or application programs.

The relationship between schemas, instances, and data independence makes a DBMS more flexible, efficient, and easy to maintain. It allows database administrators to improve storage or modify the database structure without interrupting users or applications.

Conclusion

Schemas define the structure of a database, while instances represent the current data. **Physical** and **Logical Data Independence** make the database flexible, maintainable, and efficient by allowing changes at one level without affecting other levels.

10. Design a concept map for the roles and responsibilities in a DBMS environment: DBA, End User, Application Developer.



Roles and Responsibilities in a DBMS Environment

A **Database Management System (DBMS)** involves different users who work together to manage, develop, and use the database efficiently. The three main roles are the **Database Administrator (DBA)**, **Application Developer**, and **End User**.

Database Administrator (DBA)

The DBA is responsible for managing and maintaining the database system. The DBA installs and configures the DBMS, ensures data security, performs backup and recovery, manages user access, monitors performance, and maintains data integrity.

Application Developer

The Application Developer designs and develops database applications. They create user-friendly interfaces, write SQL queries and application code, test applications, and ensure that users can access and manipulate data efficiently.

End User

The End User interacts with the database through applications. They enter, update, retrieve, and view data according to their requirements without directly managing the database.

Conclusion

The **DBA**, **Application Developer**, and **End User** work together to ensure that the database system is secure, efficient, reliable, and easy to use. Each role has specific responsibilities that contribute to the successful operation of a DBMS.